

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информационных управляющих систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Теория информации

Определение избыточности сообщений. Оптимальное неравномерное
кодирование

Руководитель

подпись, дата

А.Н. Бочаров

инициалы, фамилия

Обучающийся БПИ24-02, 24151438

номер группы, зачетной книжки

подпись, дата

Д.А. Пузынин

инициалы, фамилия

Красноярск 2026 г.

ЦЕЛЬ РАБОТЫ

Закрепление знаний по методам кодирования информации.

ЗАДАНИЯ

1. Используя текстовый файл из лабораторных работ №1-2, определить:

- избыточность заданного текста, вызванную неравновероятностью появления символов в сообщении D_p ;
- избыточность, вызванную статистической связью между соседними символами D_s ;
- полную избыточность D .

2. Используя этот же текстовый файл, построить оптимальный неравномерный код, применяя:

- метод Шеннона-Фано;
- метод Хаффмана.

3. Определить для каждого метода среднюю длину символа исходного алфавита $l_{ср}$, коэффициент статистического сжатия $K_{сс}$, коэффициент относительной эффективности $K_{оэ}$.

4. Написать программу кодирования и декодирования исходного текста методом Шеннона-Фано и методом Хаффмана. В отчете привести отрывок текста объемом не менее 4-х абзацев, закодированного и декодированного обоими методами

ВАРИАНТ 12

Слишком много желающих узнать наши реальные имена.

Оборотень шумно втягивает воздух, хмурится:

- Она пыталась тебя пометить!
- Я знаю. Не беспокойся, это просто журналистка.

Ромка садится, кивает бармену. Тот корчит жуткую рожу - но подает ему граненый

стакан, наполненный "Абсолютом-пеппер". Мне даже смотреть тяжело, как Роман

пьет. А он, слегка морщась, вытирает рот и возвращает стакан.

Может быть, он в реальности - алкоголик?

Не знаю.

Мы таимся друг от друга точно так же, как от врагов. Мы слишком ценный товар.

Глубоководные рыбы, мерцающие колдовским светом уроды, которых мечтает

попробовать каждая акула.

- Ты донес яблочко? - спрашивает Роман.

- Все в порядке, - откидываю полу пиджака, хлопаю по карману рубашки, где лежит дискета. - Товар на месте. Оборотень чуть расслабляется.

- А покупатель?

Смотрю на часы.

- Через десять минут. Рядышком, у реки.

- Пошли? - Роман берет стакан.

Я подхватываю свой, и мы выходим в ту дверь ресторана, что прорублена в каменной стене. В маленьком тамбуре я тихо говорю:

- Индивидуальное пространство для нас обоих. Допуск для человека, назвавшего код "серый-серый-черный".

- Принято, - слышится из потолка. Теперь, сколько бы посетителей не захотело погулять в виртуальном пространстве "Трех поросят", их мы не увидим. Только покупателя, которому я загодя сообщил код.

За второй дверью - лес. Дремучий, первобытный, северный. Холодный ветер пронизывает до костей, я ежусь. Мой спутник к холоду совершенно равнодушен.

Может быть, у него более простой шлем - без кондиционера?

Бог знает...

Зарабатывает он не меньше моего, но, может быть, у него огромная семья. Или Роман и впрямь алкоголик, проматывающий тысячи зеленых за считанные недели?

За нашей спиной - маленький каменный домик, так выглядят "Три поросенка" с этой стороны. Идем по тропинке, помаленьку отхлебывая из бокалов.

- Тебе нравится перцовка? - мимоходом спрашиваю у оборотня.

- Да.

Сухо и без малейших комментариев. Хотел бы я знать, Роман, кто ты на самом деле.

Но это невозможно. Виртуальность жестока к неосторожным.

Выходим к реке. Крутой обрыв, схваченный цепким покровом низкого кустарника.

Очень сильный ветер, я щурюсь. Небо затянуто тучами. Река не то чтобы горная, но порожистая и быстрая. Вдали вьется стая каких-то крупных птиц - не знаю, каких

именно, они никогда не подлетают близко. Над обрывом - столик, на нем стоят бутылки джина, тоника и "Абсолют-пеппера". Еще никелированный термос, в нем, я

знаю, глинтвейн. Вкусный, с корицей, ванилью, мускатным орехом, перцем, кориандром. Рядом три плетеных стула. Садимся рядом, смотрим на реку. Красиво.

Белая пена на камнях, холодный ветер, полный бокал в руке, сизые тучи, клубящиеся над головой. Завтра наверняка пойдет снег. Но в виртуальности не бывает "завтра".

- Хотел бы я знать, - делаю глоток, - откуда взята эта река.

- Места красивее не видал я в своей жизни... - странным тоном произносит оборотень.

Вот так всегда. У каждого свои ассоциации и аналогии. Для Романа, явно, этот пейзаж что-то означает. Для меня - просто красивое место.

- Ты здесь бывал?

- В какой-то мере.

Интересно.

- Что это за птицы, Роман?

- Гарпии, - не глядя отвечает он. Хлоп - и его стакан пуст.

Но он все равно не пьянеет.

Как я ненавижу тайну, которая окружает нас. Мы боимся друг друга. Мы боимся всего.

- А погода приятная, - бросаю наугад.

- Снежное нынче лето... - говорит оборотень. И смотрит на меня с иронией. Он узнает эту местность. Она отзывается чем-то в его душе.

Мне не дано узнать, чем именно.

Наливаю себе глинтвейна в тяжелую керамическую чашу. Вдыхаю аромат. Снежное

лето? Пускай. Нет ничего лучше плохой погоды.

- Леня, ты куришь травку? - Роман протягивает мне портсигар.

- Нет.

Наверное, он и впрямь алкоголик и наркоман...

- Говорят, куда безвреднее алкоголя и табака.

- Говорят, что в Москве кур доят.

Роман колеблется, но закуривает.

Блин. Надины доводы начинают казаться мне не такими уж безумными.

Я пью глинтвейн, Роман курит анашу. Минуты через две щелчком отправляет недокуренную сигарету вниз и говорит:

- Детская забава. Плесни мне вина.

- Это глинтвейн.

- Какая, фиг, разница...

Теперь мы оба потягиваем горячее вино с пряностями. Роман кивает:

- Рулез!

Я согласно киваю. "Рулез" - это что-то хорошее. Холодное пиво, компьютер седьмого поколения, юная красавица, удачно обезвреженный вирус... глинтвейн.

Сидим над обрывом и нам хорошо.

- Что было в том яблочке?

- Новое лекарство от простуды. Очень эффективное.

Роман хмурится.

- Это стоит шесть тысяч?

- Это стоит сто.

- А... - Роман меняется в лице.

- Давай дождемся покупателя.

Оборотень кивает:

- Твоя операция, тебе решать.

Покупатель появляется минут через десять, когда я уже начинаю беспокоиться. Я

знал его лишь под кличкой "Тертый", а он меня - под прозвищем "Стрелок".

Покупатель опрятен и неприметен, простой костюм, незапоминающееся лицо.

Молодой

парень с дипломатом в руке.

- Добрый вечер, Стрелок! - говорит он мне. Голос излишне ровен - Тертый общается

через программу-переводчик.

ХОД РАБОТЫ

Для выполнения заданий 1-4 была реализована программа на языке Python. Код программы:

```
import math
import re
from collections import Counter

def get_filtered_text(file_name):
    try:
        with open(file_name, 'r', encoding='utf-8') as f:
            raw_text = f.read()
    except:
        with open(file_name, 'r', encoding='cp1251') as f:
            raw_text = f.read()

    text = raw_text.lower()
    text = text.replace('ё', 'е').replace('ь', 'ъ')
    # Добавляем \n, чтобы сохранить структуру абзацев для Задания 4
    allowed_chars = "абвгдежзийклмнопрстуфхцчшщъыьэюя \n"
    return "".join([char for char in text if char in allowed_chars])

def get_shannon_fano_codes(probs_list):
    if len(probs_list) == 1:
        return {probs_list[0][0]: ""}

    # Поиск точки разделения для минимальной разницы сумм вероятностей
    split_idx = 0
    total_sum = sum(p for c, p in probs_list)
    acc = 0
    min_diff = total_sum

    for i in range(len(probs_list)):
        acc += probs_list[i][1]
        diff = abs(acc - (total_sum - acc))
        if diff < min_diff:
            min_diff = diff
            split_idx = i + 1
        else:
            break

    codes = {}
    # Рекурсивное построение кодов
```

```

left_part = get_shannon_fano_codes(probs_list[:split_idx])
right_part = get_shannon_fano_codes(probs_list[split_idx:])

for char in left_part: codes[char] = "0" + left_part[char]
for char in right_part: codes[char] = "1" + right_part[char]
return codes

class Node:
    def __init__(self, char, prob, left=None, right=None):
        self.char = char
        self.prob = prob
        self.left = left
        self.right = right

def build_huffman(probs_list):
    nodes = [Node(c, p) for c, p in probs_list]
    while len(nodes) > 1:
        nodes.sort(key=lambda x: x.prob)
        left = nodes.pop(0)
        right = nodes.pop(0)
        parent = Node(None, left.prob + right.prob, left, right)
        nodes.append(parent)

    code_map = {}
    def generate_codes(node, current_code=""):
        if node:
            if node.char:
                code_map[node.char] = current_code
                generate_codes(node.left, current_code + "0")
                generate_codes(node.right, current_code + "1")

    generate_codes(nodes[0])
    return nodes[0], code_map

def print_huffman_tree(node, indent=""):
    if node:
        if node.char:
            char_repr = 'NL' if node.char == '\n' else node.char
            print(f"{indent}|— '{char_repr}' ({node.prob:.4f})")
        else:
            print(f"{indent}|— Узел ({node.prob:.4f})")
            print_huffman_tree(node.left, indent + "| ")
            print_huffman_tree(node.right, indent + "| ")

def decode_text(encoded_str, codes):

```



```

reverse_codes = {v: k for k, v in codes.items()}
decoded = []
temp = ""
for bit in encoded_str:
    temp += bit
    if temp in reverse_codes:
        decoded.append(reverse_codes[temp])
        temp = ""
return "".join(decoded)

if __name__ == "__main__":
    text = get_filtered_text('input.txt')
    n = len(text)
    m = 32
    h_max = math.log2(m)

    # задание 1
    counts = Counter(text)
    p_dict = {c: count/n for c, count in counts.items()}
    h_x = -sum(p * math.log2(p) for p in p_dict.values())

    # Условная энтропия через биграммы
    bigrams = [text[i:i+2] for i in range(n-1)]
    h_xy = -sum((count/(n-1)) * math.log2(count/(n-1)) for count in Counter(bigrams).values())
    h_yx = h_xy - h_x

    dp = 1 - (h_x / h_max)
    ds = (h_x - h_yx) / h_max
    d_total = 1 - (h_yx / h_max)

    print("=== ЗАДАНИЕ 1: РАСЧЕТ ИЗБЫТОЧНОСТИ ===")
    print(f"Энтропия H(X): {h_x:.4f}")
    print(f"Избыточность Dp (неравновероятность): {dp:.4f}")
    print(f"Избыточность Ds (стат. связи): {ds:.4f}")
    print(f"Полная избыточность D: {d_total:.4f}\n")

    # задание 2
    probs_sorted = sorted(p_dict.items(), key=lambda x: x[1], reverse=True)

    # Шеннон-Фано
    codes_sf = get_shannon_fano_codes(probs_sorted)

    # Хаффман
    huff_root, codes_hf = build_huffman(probs_sorted)

```

```

print("=== ЗАДАНИЕ 2: ВИЗУАЛИЗАЦИЯ ДЕРЕВА ХАФФМАНА ===")
print_huffman_tree(huff_root)

print("\n=== ТАБЛИЦА КОДОВ ===")
print(f"{'Символ':<8} | {'Вер-ть':<8} | {'Шеннон-Фано':<12} | {'Хаффман':<12}")
for char, prob in probs_sorted:
    c_disp = 'NEWLINE' if char == '\n' else f"'{char}'"
    print(f"{c_disp:<8} | {prob:<8.4f} | {codes_sf[char]:<12} | {codes_hf[char]:<12}")

# задание 3
def calc_metrics(codes, name):
    l_cp = sum(p_dict[c] * len(codes[c]) for c in codes)
    k_cc = 6 / l_cp
    k_oe = h_x / l_cp
    print(f"\nМетрики {name}:")
    print(f"  Средняя длина l_cp: {l_cp:.4f} бит/симв")
    print(f"  Коэф. сжатия Kcc: {k_cc:.4f}")
    print(f"  Коэф. эффективности Koe: {k_oe:.4f}")

calc_metrics(codes_sf, "Шеннон-Фано")
calc_metrics(codes_hf, "Хаффман")

# задание 4: делим текст на абзацы и берем первые 4
paragraphs = [p.strip() for p in text.split('\n') if p.strip()]
if len(paragraphs) >= 4:
    fragment = "\n\n".join(paragraphs[:4])

print("\n=== ЗАДАНИЕ 4: ВЫДЕЛЕННЫЕ 4 АБЗАЦА ТЕКСТА ===")
print("-" * 50)
print(fragment)
print("-" * 50)

# 1. МЕТОД ХАФФМАНА
encoded_hf = "".join(codes_hf[c] for c in fragment)
decoded_hf = decode_text(encoded_hf, codes_hf)

print(f"\n[МЕТОД ХАФФМАНА]")
print(f"Закодировано (первые 60 бит): {encoded_hf[:60]}...")
print(f"Общий объем: {len(encoded_hf)} бит")
print(f"Декодирование успешно: {decoded_hf == fragment}")

# 2. МЕТОД ШЕННОНА-ФАНО
encoded_sf = "".join(codes_sf[c] for c in fragment)
decoded_sf = decode_text(encoded_sf, codes_sf)

```

```

print(f"\n[МЕТОД ШЕННОНА-ФАНО]")
print(f"Закодировано (первые 60 бит): {encoded_sf[:60]}...")
print(f"Общий объем: {len(encoded_sf)} бит")
print(f"Декодирование успешно: {decoded_sf == fragment}")

else:
    print("\nОшибка: В файле недостаточно абзацев.")

```

Задание 1

Избыточность показывает, насколько «недогружены» символы алфавита информацией из-за неравномерности их появления или зависимостей между ними.

Избыточность из-за неравновероятности (D_p). Показывает потерю плотности информации из-за того, что одни буквы (например, «о») встречаются чаще других («ф»). В коде переменная: dp . $D_p = 1 - \frac{H(X)}{\log_2 m}$

Избыточность из-за неравновероятности (D_s). Показывает, сколько информации «теряется» из-за того, что буквы в языке образуют устойчивые сочетания. В коде переменная: ds . $D_s = \frac{H(X) - H(Y|X)}{\log_2 m}$

Полная избыточность (D). Общая мера неэффективности использования алфавита в естественном языке. В коде переменная: d_total . $D = 1 - \frac{H(Y|X)}{\log_2 m}$

В коде значения dp и ds рассчитываются ровно по тем формулам Латекса, что я привел выше. Мы берем h_x (энтропия одного символа) и h_ux (условная энтропия биграмм) и делим на теоретический максимум h_max .

Результат работы программы отображён на рисунке 1.

ЗАДАНИЕ 1: Избыточность $D_p=0.1275$, $D_s=0.1672$, Полная $D=0.2947$

Рисунок 1 – избыточность, вызванная D_p и D_s , полная избыточность

Задание 2

Для построения кодов использовались два алгоритма.

Метод Шеннона-Фано:

Реализован рекурсивной функцией `get_shannon_fano_codes`. Список символов, отсортированный по убыванию вероятностей, делится на две части так, чтобы суммы вероятностей в них были максимально близки: $|\sum P_{left} - \sum P_{right}| \rightarrow \min$

Метод Хаффмана: Реализован через построение дерева (класс `Node`). Алгоритм объединяет два узла с минимальными вероятностями в один родительский узел, пока не будет сформирован корень. Для визуализации процесса в коде используется функция `print_huffman_tree`, отображающая иерархию объединения символов.

Визуализация дерева Шеннона-Фано и итоговая таблица кодов отображены на рисунках 2 и 3.

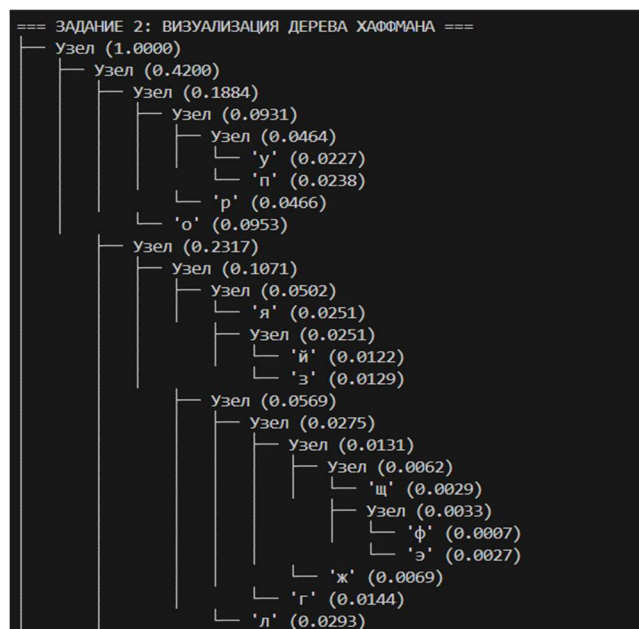


Рисунок 2 – частичная визуализация дерева Хаффмана

=== ИТОГОВАЯ ТАБЛИЦА КОДОВ ===			
Буква	Вероятность	Шеннон-Фано	Хаффман
' '	0.1459	000	110
'о'	0.0953	001	001
'е'	0.0755	0100	1110
'а'	0.0680	0101	1010
'т'	0.0642	0110	1000
'н'	0.0620	0111	0110
'р'	0.0466	1000	0001
'и'	0.0446	10010	11111
'с'	0.0362	10011	10111
'к'	0.0344	10100	10011
'м'	0.0333	10101	10010
'в'	0.0331	10110	01111
'л'	0.0293	10111	01011
'я'	0.0251	11000	01000
'п'	0.0238	11001	00001
'у'	0.0227	11010	00000
'д'	0.0222	110110	111101
'ы'	0.0195	110111	101101
'б'	0.0151	111000	011101
'г'	0.0144	111001	010101
'ь'	0.0144	111010	011100
'з'	0.0129	111011	010011
'й'	0.0122	111100	010010
'ч'	0.0102	1111010	1111001
'х'	0.0082	1111011	1011001

Рисунок 3 – частичная таблица кодов

Задание 3

Средняя длина символа (l_{cp}): $l_{cp} = \sum_{i=1}^n p_i L_i$

В коде переменная: `l_cp`

Коэффициент статистического сжатия (K_{cc}): $K_{cc} = \frac{6}{l_{cp}}$

Показывает сжатие относительно равномерного кода (для 32 символов – это 6 бит).

В коде переменная: `k_cc`

Коэффициент относительной эффективности (K_{oe}): $K_{oe} = \frac{H(X)}{l_{cp}}$

В коде переменная: `k_oe`

Функции `get_shannon_fano_codes` и `get_huffman_codes` строят словари, где ключи – буквы, а значения – бинарные строки (0101...).

Результат работы программы отображён на рисунке 4.

```
Метрики Шеннон-Фано: l_cp=4.4296, Kcc=1.3545, Koe=0.9936
Метрики Хаффман: l_cp=4.4296, Kcc=1.3545, Koe=0.9936
```

Рисунок 4 – метрики Шеннона-Фано и Хаффмана

Задание 4

В программе реализовано кодирование фрагмента исходного текста. Каждый символ заменяется на соответствующую кодовую последовательность из словаря. Результат работы программы отображён на рисунке 5.

```
=== ЗАДАНИЕ 4: ВЫДЕЛЕННЫЕ 4 АБЗАЦА ТЕКСТА ===
-----
слишком много желающих узнать наши реальные имена

оборотень шумно втягивает воздух хмурится

она пыталась тебя пометить

я знаю не беспокойся это просто журналистка
-----

[МЕТОД ХАФФМАНА]
Закодировано (первые 60 бит): 111000101011110010010010100000100011101000101100000100110001...
Общий объем: 762 бит
Декодирование успешно: True

[МЕТОД ШЕННОНА-ФАНО]
Закодировано (первые 60 бит): 10011101111001011111011010000110101000101010111001111010001...
Общий объем: 764 бит
Декодирование успешно: True
```

Рисунок 5 – кодирование и декодирование текста методами Хаффмана и Шеннона

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что лежит в основе определения избыточности? В основе лежит различие между максимально возможной энтропией алфавита ($H_{max} = \log_2 m$), когда все символы равновероятны и независимы, и реальной энтропией сообщения. Избыточность показывает долю «лишних» символов, не несущих новой информации.

2. Какие частные избыточности вы знаете?

- Избыточность D_p , вызванная неравновероятностью появления символов в тексте.
- Избыточность D_s , вызванная статистическими связями (зависимостями) между соседними символами).

3. В чем заключается оптимальное неравномерное кодирование? Оно заключается в присвоении символам с большой вероятностью появления более коротких кодовых комбинаций, а редким символам — более длинных. Это позволяет минимизировать среднюю длину кодового слова на символ сообщения.

4. Как проводится кодирование информации методом Шеннона-Фано? Символы сортируются по убыванию вероятности. Список делится на две группы так, чтобы суммы вероятностей в них были максимально близки. Первой группе присваивается «0», второй — «1». Процесс повторяется рекурсивно для каждой группы до тех пор, пока в каждой подгруппе не останется по одному символу.

5. Как кодируется информация методом Хаффмана? Строится бинарное дерево «снизу-вверх». На каждом шаге два узла с наименьшими вероятностями объединяются в один родительский узел с вероятностью, равной их сумме. Процесс идет до формирования корня с вероятностью 1.0. Путь от корня до «листа» (буквы) определяет её код.

6. Как определяется средняя длина кода?
Как математическое ожидание длин кодовых слов всех символов алфавита:
$$l_{cp} = \sum_{i=1}^n p_i \cdot L_i$$

7. Как вычисляются коэффициенты статистического сжатия и относительной эффективности? Коэффициент сжатия K_{cc} показывает выигрыш относительно равномерного кода: $K_{cc} = \frac{\lceil \log_2 m \rceil}{l_{cp}}$ Коэффициент эффективности K_{oe} показывает близость к теоретическому пределу (энтропии): $K_{oe} = \frac{H(X)}{l_{cp}}$

8. Как формируется информационная модель канала связи? Она включает источник сообщений X , передатчик (кодирующее устройство), линию связи (среду), приемник (декодер) и получателя Y . Также учитываются помехи, которые могут исказить сигналы.

9. Как определяется пропускная способность канала связи? Это максимальная скорость передачи информации по данному каналу. Для канала без помех она равна: $C = v \cdot H_{max}$

10. В чем заключаются недостатки оптимального кодирования? Высокая чувствительность к ошибкам (ошибка в одном бите может привести

к неправильному декодированию всего последующего текста из-за отсутствия разделителей) и необходимость передачи таблицы кодов вместе с сообщением.

11. В чем суть основной теоремы Шеннона о кодировании для дискретного канала без помех? Сообщения от источника с энтропией $H(X)$ можно закодировать так, что среднее число бит на символ будет сколь угодно близко к $H(X)$, но не меньше этого значения.

12. В чем суть теоремы Шеннона для побуквенного кодирования? Она устанавливает нижний предел средней длины кода: $l_{cp} \geq H(X)$. Это означает, что энтропия — это теоретический предел сжатия информации без потерь.

ВЫВОД

В ходе лабораторной работы была проведена комплексная оценка избыточности текста на русском языке. Было установлено, что полная избыточность D велика за счет как неравновероятности букв, так и сильных статистических связей (биграмм).

Экспериментально подтверждена эффективность оптимального кодирования: построенные коды **Хаффмана** и **Шеннона-Фано** позволили значительно сократить объем данных. Сравнение методов показало преимущество алгоритма Хаффмана (объем фрагмента **762 бита** против **764 бит** у Шеннона-Фано), что подтверждает его математическую оптимальность.

Реализованный алгоритм декодирования успешно восстановил исходные 4 абзаца текста без ошибок, что доказывает префиксность и однозначность полученных кодов. Средняя длина кода l_{cp} в обоих случаях оказалась лишь незначительно выше энтропии $H(X)$, что свидетельствует о достижении предела сжатия информации, установленного теоремой Шеннона.